SDN + SOCKET PROGRAMMING · PROJECT 20

Adaptive Load Balancer

Deliverable 1: TCP sockets, Mininet and a Ryu controller working together

Leshawn Arron Lewis · Kishor Kumar M · Dhule Shashwati Vijay

Team 9 · Section E · Jackfruit Mini Project

THE PROBLEM

Many servers, one address: who gets the next connection?

Why it matters

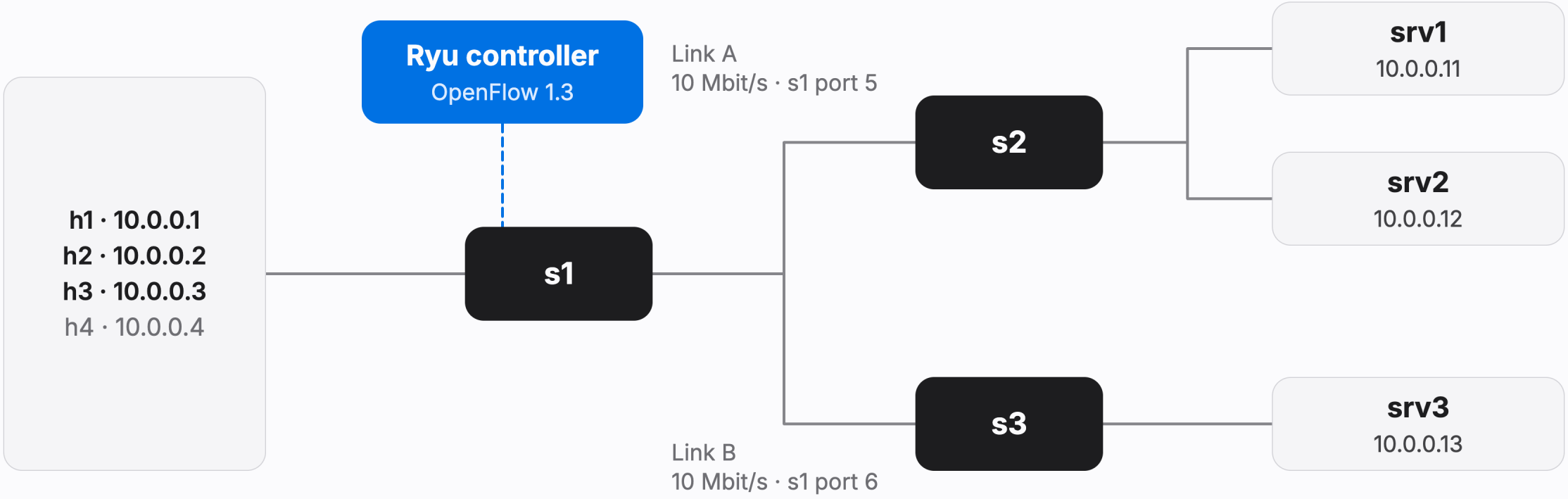
- Clients only know one address: 10.0.0.100:8080
- Three TCP servers sit behind it
- A fixed rule like round-robin ignores how busy each server is
- A busy or slow server keeps getting work

Deliverable 1 goals

- TCP client/server with concurrent clients
- Mininet topology run by a Ryu controller
- Servers report their load; the controller uses it to choose
- An end-to-end demo with test cases

TOPOLOGY

Three switches, four clients, three servers



Clients share the virtual address 10.0.0.100:8080. srv1 and srv2 share link A; srv3 is alone on link B. Ryu controls all three switches; h4 is idle in D1.

COMPONENTS

Five programs and one README

File	Role	Key parts
server.py	TCP server on srv1–srv3	thread per connection, JSON request/reply, UDP load reporter
client.py	TCP client on h1–h3	parallel threads, a new connection per request, reply tally
controller.py	Ryu SDN controller	ARP for the VIP, load reports, server choice, flow rules
topo.py	Mininet network + demo	builds the topology, starts everything, prints PASS/FAIL
test_socket.py	Socket test	TCP replies and UDP report checked without Mininet
README.md	Documentation	architecture, flow, how to run, test cases

SOCKET IMPLEMENTATION

TCP for requests, UDP for load reports

TCP server

- socket(AF_INET, SOCK_STREAM), port 8080
- One thread per accepted connection
- Reads {"id": n}, replies {"id": n, "server": "srv1"}
- 5 s timeout; bad JSON gets an error reply

TCP client

- 4 threads per host run in parallel
- Each request opens a new connection
- So the controller decides per connection
- Counts which server answered

UDP load report

- socket(AF_INET, SOCK_DGRAM)
- Every 0.5 s: {"srv": "srv1", "active": 2}
- Sent to 10.0.0.254:9999, an address no host owns
- Small, frequent, loss-tolerant

SOCKET–SDN INTEGRATION

How one request travels

STEP 1

Servers report load

A rule on every switch hands the UDP reports to Ryu as PacketIns.

STEP 2

Client resolves the VIP

h1 asks who has 10.0.0.100; Ryu answers the ARP itself.

STEP 3

SYN reaches s1

No rule matches yet, so s1 sends the first packet to Ryu.

STEP 4

Ryu picks a server

Lowest reported active connections plus those sent since that report.

STEP 5

Ryu installs two rules

Forward to the server over link A or B; rewrite replies to come from the VIP.

STEP 6

Switches do the rest

Later packets never reach Ryu; s2 and s3 forward as learning switches.

FLOW RULES

What the controller installs on s1

Priority	Match	Action	When
0	anything (table-miss)	send to controller	switch connects
100	UDP to 10.0.0.254:9999	send to controller	switch connects
10	client → 10.0.0.100:8080, one TCP connection	set dst MAC and IP to the server, out port 5 or 6	first SYN
10	server:8080 → that client	set src MAC and IP to the VIP, out the client port	first SYN
1	known destination MAC	out the learned port	learning switch

NAT rules expire after 10 s idle. A repeated SYN within those 10 s goes to the same server, so a connection is never split.

INITIAL TESTING

Five test cases

Test	How	Pass when
TCP request/reply with concurrency	test_socket.py	4 threads × 5 requests all answered
UDP load report format	test_socket.py	report arrives as valid JSON
Connectivity through the switches	ping h1 → srv2, pingall	0% packet loss
Load balancing end to end	topo.py: 3 hosts × 4 threads × 10	no errors; every server answers
Load reports change decisions	topo.py: srv1 made 6× slower	srv1 receives the fewest requests

DEMO

What we will run

`python3 test_socket.py`	Sockets work: PASS TCP, PASS UDP
`sudo RYU=… python3 topo.py`	Network, controller and balancing: RESULT PASS
`mininet> pingall`	Every host reaches every other host
`mininet> net`	Topology built as designed
`mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1`	Rules the controller installed
`mininet> sh tail -5 controller.log`	Each connection and the server chosen

DELIVERABLE 2

From fewest connections to an adaptive, link-aware balancer

The design for the rest of the project, and what we expect it to show

D2 DESIGN

What D2 adds on top of D1

Feature	D1 (built)	D2 (design)
Decision rule	fewest connections	weighted score: load, CPU, link use
Server report	{srv, active}	{srv, active, cap, cpu, done} every 0.25 s
Network monitoring	none	port counters on s1 every 1 s → link A and B use
Overload	not handled	drain at ≥ 90% load, resume below 70%
Server failure	not handled	down after 3 s without a report; its rules deleted
Baseline	none	static round-robin mode for comparison
Evaluation	demo PASS/FAIL	six experiments, static vs adaptive

D2 DESIGN

The adaptive score: lowest wins

$$\text{score} = 0.5 \times \text{load} + 0.2 \times \text{CPU} + 0.3 \times \text{link use}$$

load

Estimated connections ÷ capacity. Estimate = reported active + new since the report − completion rate × time since the report.

CPU

Reported by each server, from 0 to 1.

link use

Traffic on that server's link (A or B) ÷ 10 Mbit/s, from the switch port counters.

D2 DESIGN

Server states and monitoring

Healthy

When: load below 90%

Effect: can win new connections

Draining

When: load ≥ 90%, until it drops below 70%

Effect: gets no new connections; existing ones finish

Down

When: no report for 3 s

Effect: skipped; its flow rules are deleted so clients reconnect elsewhere

Two thresholds (90% and 70%) stop a server flipping in and out of service. Port stats from s1 are polled every second to measure links A and B.

D2 EXPERIMENTS · 1 OF 2 Predicted, not measured

Each scenario runs in static and adaptive mode

Experiment	What changes	What we expect
E1 Balanced	3 equal servers, normal load	Prediction: both modes give about a third to each server; adaptive should not be worse
E2 Unequal servers	srv3 has 2 workers and runs 2× slower	Prediction: static still sends srv3 a third and requests queue; adaptive sends it far less, so throughput rises and p95 drops
E3 Hot server	h4 floods srv1 directly from one third of the run	Prediction: static keeps feeding srv1 and latency climbs; adaptive moves new connections to srv2 and srv3

D2 EXPERIMENTS · 2 OF 2 Predicted, not measured

Failure, a busy link, and scaling

Experiment	What changes	What we expect
E4 Server failure	srv2 is killed one third of the way in	Prediction: static keeps sending srv2 a third of requests, which fail until the run ends; adaptive stops within about 3 s
E5 Busy link	9 Mbit/s iperf traffic fills link A	Prediction: adaptive sends more than a third to srv3 on link B; static stays at a third each
E6 Scaling	1, 2, 4 and 8 client threads per host	Prediction: latency rises with load in both modes; with equal servers the two curves stay close

D2 EVALUATION

What we will measure and how

Throughput
successful requests per second

p95 latency
time within which 95% of requests finish

Error rate
share of requests that failed

Server share
fraction of requests each server handled

Fairness
Jain’s index over the server shares

Recovery
how long errors last after a failure

Method: each scenario runs 30 s in static mode, then 30 s in adaptive mode, with clients h1–h3 at 4 threads each. The event (slowdown, flood, failure or iperf) starts one third of the way in.

OUR HYPOTHESIS Prediction

Adaptive should match static when everything is equal, and beat it whenever a server or link is slow, overloaded or failing.

D2 will test this with the six experiments and report the measured numbers.

Thank you

Questions?

Team 9 · Leshawn Arron Lewis · Kishor Kumar M · Dhule Shashwati Vijay